Jev and the cost of an agent's decisions

Routing, guardrails, policy checks and loop control priced on OpenAI, Anthropic and Google models and on TypeSafe's Jev, against naive, cached and tuned stacks. With the reference architecture, the method, and a calculator anyone can run.

$315

a month for all 5 questions on Jev at 100,000 requests a day

2.5× to 133×

cheaper decision layer than a tuned baseline, by model

1 to 68%

off the whole bill, from tuned stacks to naive ones

Gabriel Ohaike

Enterprise Principal Agentic AI Architect

No affiliation with or compensation from TypeSafe. Views my own.

Prices as of 12 October 2026

SUMMARY

Jev's price advantage is real, and its size depends on your stack

Before an agent writes anything it answers a handful of small questions: where to route, whether the input is safe, whether the action is allowed, and what to do next. This benchmark prices that decision layer at list price on nine models from three providers, under three baselines from naive to tuned, and prices the same layer on Jev, a model that answers typed questions with calibrated probabilities and writes no text.

$90,000

a month on naive decision calls with Claude Sonnet 5.5, against $42,000 on replies

2.5× to 133×

cheaper decision layer on Jev than a tuned, cached, batched baseline

1 to 68%

off the whole bill, depending on how tuned the stack already is

Findings

1. In a naive stack the four decision calls cost more than the reply. On Claude Sonnet 5.5 they are $90,000 a month against $42,000 for replies.
2. Against a tuned baseline, with one batched call per request, a cached static prompt and terse output, Jev's decision layer is 2.5× cheaper than Claude Haiku 5.5 and 133× cheaper than GPT-5.6 Sol.
3. The saving on the whole bill depends on how tuned the stack already is: 67 to 68% for naive mid-tier stacks, 7 to 41% for typical cached small-model stacks, and 1 to 17% for tuned ones.
4. Where Jev barely wins: a tuned stack on Claude Haiku 5.5 saves $478 a month. There the case for Jev is calibrated confidence and one fast pass, and price matters little.
5. Jev belongs in the architecture as the judge. Code and systems of record decide what its facts permit, OPA or Cedar enforces, and the LLM writes.

Scope. This benchmark measures list-price token cost. It does not measure accuracy, which depends on your decisions and data and needs a shadow run on your own traffic (section 5). The author has no affiliation with or compensation from TypeSafe.

SECTION 1

1 Method

The reference workload is an enterprise agent handling 100,000 requests a day for 30 days. Each request answers 5 questions (route, prompt injection, personal data, policy, next step) and writes one reply. All parameters sit in one block at the top of the calculator.

Parameter	Value	Note
Static prompt per decision call	1,500 tokens	Instructions and rubric; cacheable
Request context	1,500 tokens	Changes every request; never cached
Output per separate call	150 tokens	JSON label and a short reason, thinking off
Output per question, batched	30 tokens	Terse JSON
Jev question	200 tokens	Instructions and options; Jev needs no separate static prompt
Cache hit rate	95%	Misses pay the provider's cache write price
Reply	4,000 in, 600 out	Same with and without Jev; cached in scenarios B and C

Three baselines

Baseline	How the decisions are made
Naive	4 separate calls (the guardrail covers injection and PII), no caching
Cached	The same 4 calls with each static prompt cached (equal to naive on Google, see prices)
Tuned	One batched call answering all 5 questions, static prompt cached, terse JSON

Prices, $ per 1M tokens

Model	Provider	Input	Cache hit	Cache miss	Output
GPT-5.6 Sol	OpenAI	$5.00	$0.500	$6.250	$30.00
GPT-5.6 Terra	OpenAI	$2.00	$0.200	$2.500	$12.00
GPT-5.6 Luna	OpenAI	$0.20	$0.020	$0.250	$1.20
Claude Opus 5.5	Anthropic	$4.00	$0.200	$5.000	$20.00
Claude Sonnet 5.5	Anthropic	$2.00	$0.100	$2.500	$10.00
Claude Haiku 5.5	Anthropic	$0.10	$0.010	$0.125	$0.50
Gemini 3.1 Pro Preview	Google	$2.00	$0.200	$2.000	$12.00
Gemini 3.8 Flash	Google	$0.75	$0.075	$0.750	$3.75
Gemini 3.1 Flash-Lite	Google	$0.25	$0.025	$0.250	$1.50
Jev 1.13	TypeSafe	**$0.042**	none	none	**free**

Standard list prices read from each provider's page on 12 October 2026. A cache miss on the static prefix pays the provider's cache write price: OpenAI and Anthropic both charge 1.25× input (Anthropic's five-minute cache); Google charges the input price. Caching applies only when the static prefix reaches the provider's minimum: 1,024 tokens on OpenAI, 512 on Anthropic and 4,096 on Google, from its implicit-caching table for Gemini 3.1 Pro Preview and 3.8 Flash, assumed the same for Flash-Lite. Every Google prefix in this workload is shorter, so Google runs uncached throughout and its cache storage charge never applies. Excluded: batch APIs (50% off but asynchronous, so unsuited to live decisions), volume discounts and thinking tokens. Gemini 3.8 Flash is at its introductory rate, which doubles on 1 January 2027.

SECTION 2

2 Reference architectures

Both stacks handle the same request and write the same reply. What changes is how the decisions are made and where the authority to act sits.

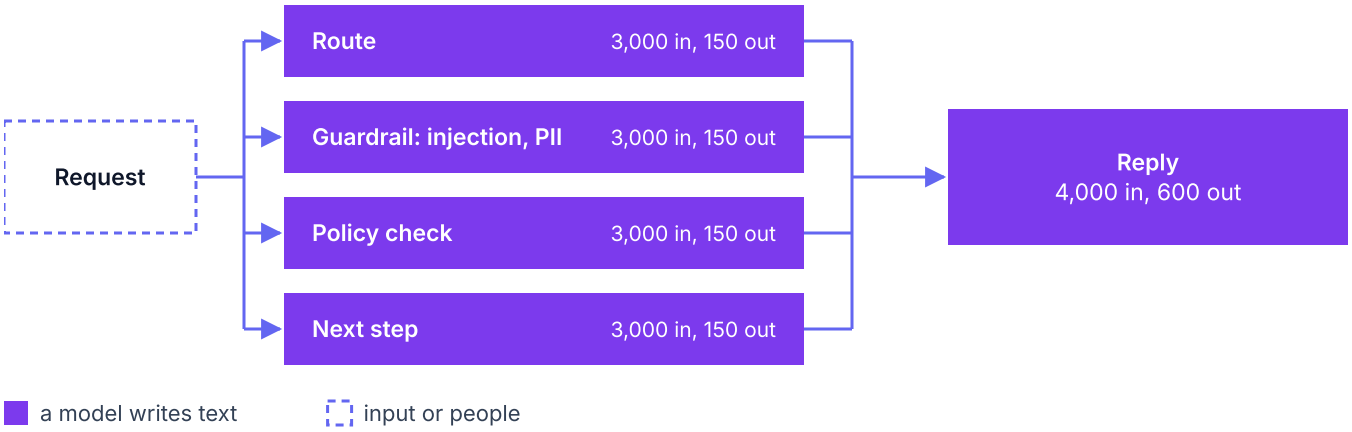


Figure 1. Without Jev. Five model calls per request. The four decision calls each read a static prompt and the request; only the reply is text a person reads.

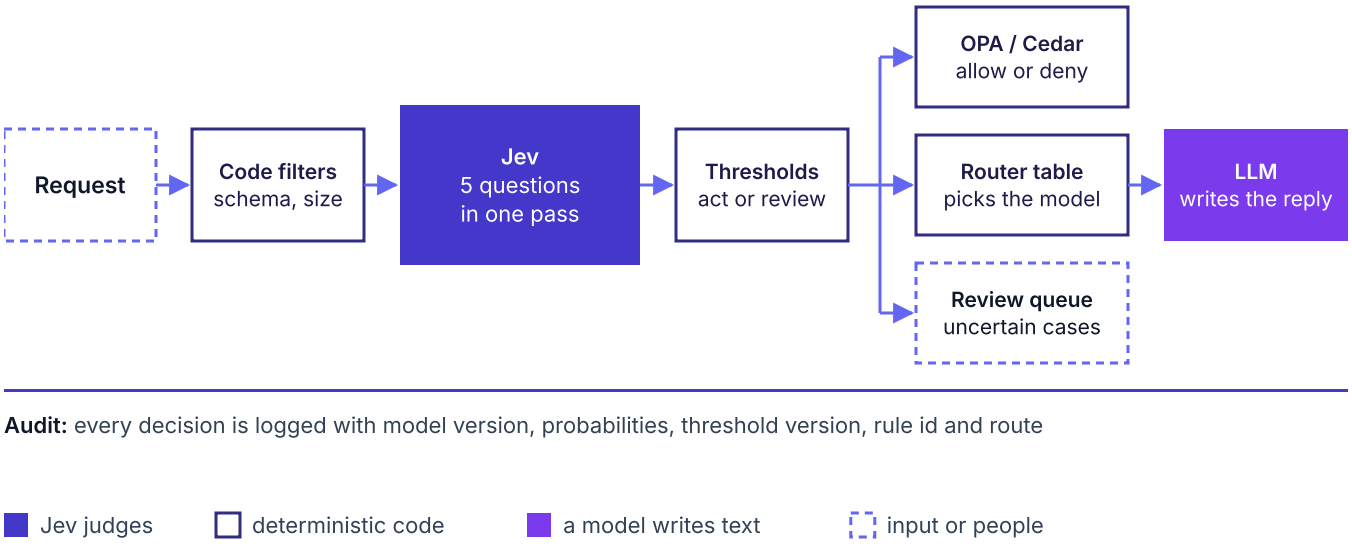


Figure 2. With Jev. Code filters first. Jev answers all 5 questions in one pass with calibrated probabilities. Thresholds in code turn probabilities into actions, OPA or Cedar enforces policy, the router table picks the model, uncertain cases go to a person, and every decision is logged.

A probabilistic judge reports facts with a confidence, and deterministic code decides what those facts permit. Every action stays reproducible and auditable while the judgment becomes cheap.

SECTION 3

3 Results: the decision layer

Monthly cost of answering the 5 questions for 100,000 requests a day. The chart shows the tuned baseline, the hardest one for Jev to beat.

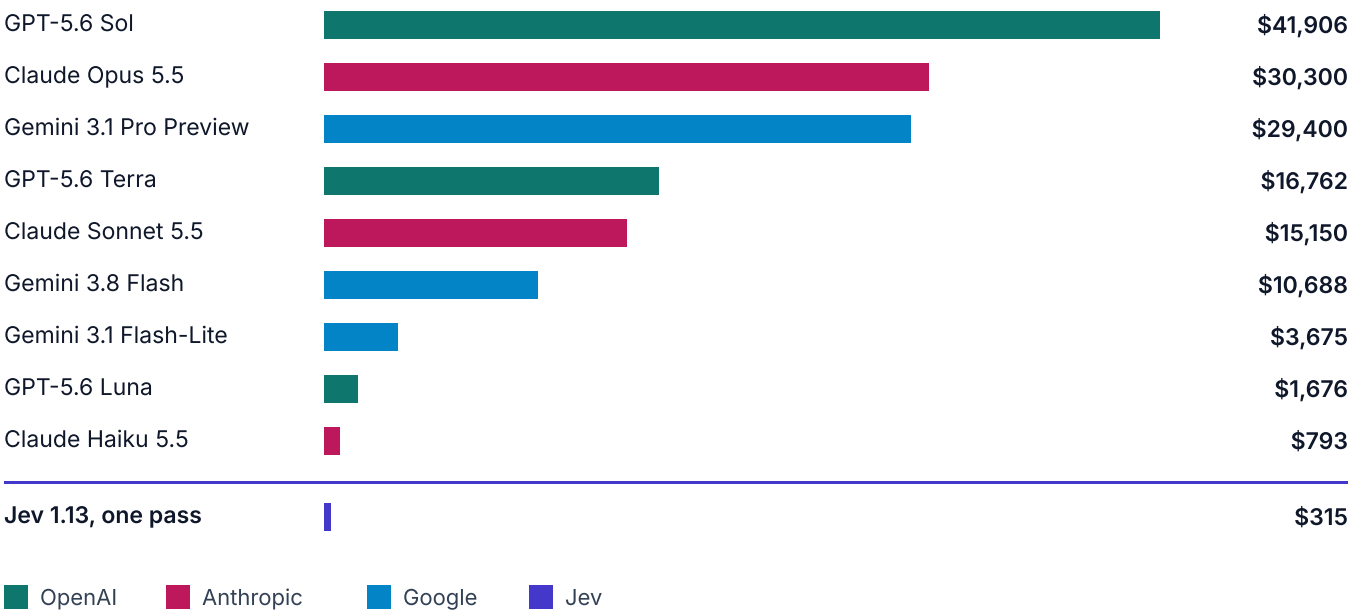


Figure 3. Tuned decision layer per month, sorted by cost. Jev's bar is drawn at its true scale.

Model	Provider	Naive	Cached	Tuned	Jev	Tuned ÷ Jev
GPT-5.6 Sol	OpenAI	$234,000	$158,175	$41,906	$315	133×
Claude Opus 5.5	Anthropic	$180,000	$115,920	$30,300	$315	96.2×
Gemini 3.1 Pro Preview	Google	$93,600	$93,600	$29,400	$315	93.3×
GPT-5.6 Terra	OpenAI	$93,600	$63,270	$16,762	$315	53.2×
Claude Sonnet 5.5	Anthropic	$90,000	$57,960	$15,150	$315	48.1×
Gemini 3.8 Flash	Google	$33,750	$33,750	$10,688	$315	33.9×
Gemini 3.1 Flash-Lite	Google	$11,700	$11,700	$3,675	$315	11.7×
GPT-5.6 Luna	OpenAI	$9,360	$6,327	$1,676	$315	5.3×
Claude Haiku 5.5	Anthropic	$4,500	$2,984	$793	$315	2.5×

Naive separate calls cost 3.2 to 5.9 times the tuned figure on every model.

SECTION 4

4 Results: the whole stack

The decision layer plus the reply, which always runs on the provider's mid-tier model. Jev replaces the decisions and leaves the reply untouched.

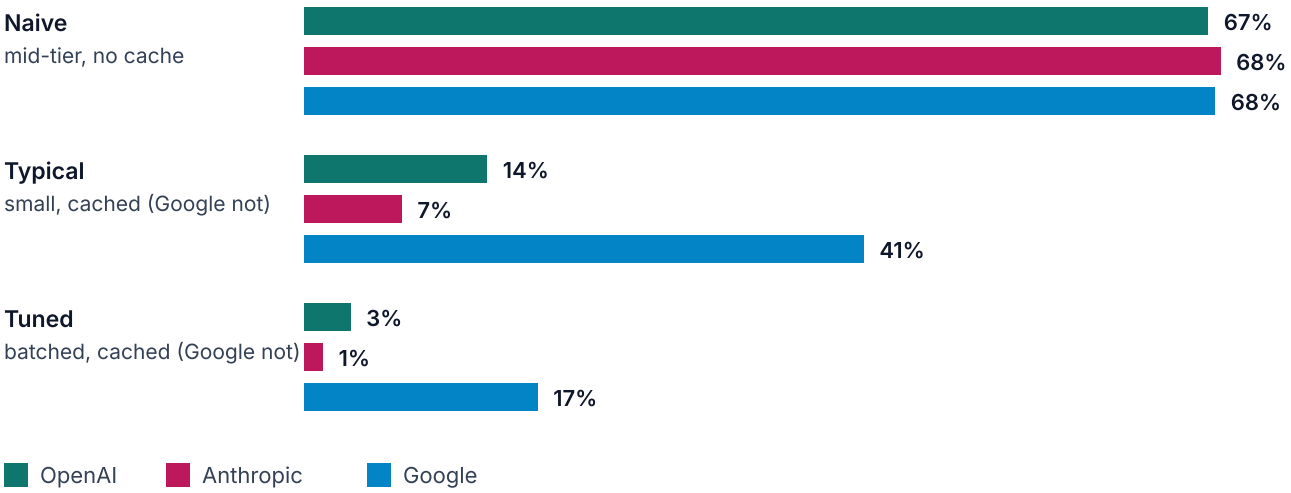


Figure 4. Share of the monthly bill saved with Jev, by scenario and provider.

Scenario	Provider · decision model	Without Jev	With Jev	Saved	%
Naive	OpenAI · GPT-5.6 Terra	$139,200	$45,915	$93,285	**67%**
Naive	Anthropic · Claude Sonnet 5.5	$132,000	$42,315	$89,685	**68%**
Naive	Google · Gemini 3.8 Flash	$49,500	$16,065	$33,435	**68%**
Typical	OpenAI · GPT-5.6 Luna	$44,344	$38,332	$6,012	**14%**
Typical	Anthropic · Claude Haiku 5.5	$36,974	$34,305	$2,668	**7%**
Typical	Google · Gemini 3.1 Flash-Lite	$27,450	$16,065	$11,385	**41%**
Tuned	OpenAI · GPT-5.6 Luna	$39,694	$38,332	$1,361	**3%**
Tuned	Anthropic · Claude Haiku 5.5	$34,783	$34,305	$478	**1%**
Tuned	Google · Gemini 3.1 Flash-Lite	$19,425	$16,065	$3,360	**17%**

Where Jev barely wins

A stack that already batches its decisions on Claude Haiku 5.5 with caching saves $478 a month, 1% of the bill. At that point the reasons to adopt Jev are calibrated confidence, which turns every threshold into a dial, and one pass for every decision, which TypeSafe measures at 70 to 500 ms end to end.

Jev billing, measured

TypeSafe's documentation says Jev reads the state once and answers every question against it, and the probe in section 7 confirmed it on 12 October 2026. The same state of about 2,000 tokens used 2,011 input tokens with one question and 2,133 with five, about 30 tokens per extra question. This benchmark therefore bills the state once per request: $315 a month. It also assumes 200 tokens per question, more than the probe measured, so the Jev figure is conservative.

SECTION 5

5 Architecture notes

Model routers

JEV DECIDES

- What kind of request is this?
- How hard is it?
- Which agent or skill owns it?

CODE KEEPS

- Price and latency per model
- Win rate on your own traffic
- Fallback order

TypeSafe documents that Jev 1.13 is weak at numeric comparison. Prices, latencies and win rates are numbers, so they live in a table the router reads, and Jev supplies only the class and difficulty of the request.

Policy engines

TypeSafe's documentation states that adversarial text in the state can move Jev's answer. So a Jev score may only tighten a decision. Ownership, amount and card history come from systems of record, and the rule allows a refund only when every one of them agrees.

```
package refunds

default allow := false
allow if {
    input.order.owner == input.user.id          # system of record
    input.order.amount <= 500                   # system of record
    input.account.card_added_last_24h == false  # system of record
    input.facts.intent == "refund"              # Jev
    input.facts.injection < 0.2                 # Jev, blocks only
}
```

Test case, run on OPA 1.4.2	allow
Attack: injection 0.97, card added today	`false`
Clean refund by the owner, older card	`true`
Jev fooled (injection 0.01), card added today	`false`
Jev fooled (injection 0.01), requester is not the owner	`false`
Jev fooled (injection 0.01), amount over the limit	`false`
Jev fooled (injection 0.01), card field missing	`false`

Thresholds and rollout

Run Jev in shadow beside the current path for two to four weeks. Measure agreement with today's answers by confidence band, and label a sample by hand to measure true accuracy, since agreement with today's system is only a proxy. Let Jev decide in the bands that clear an agreement bar you set; the author starts at 98%. Keep the old path below the bar and log every route with its threshold version.

SECTION 6

6 Limitations

Of Jev 1.13, from TypeSafe's documentation

Limitation	Design response
Unreliable at arithmetic, counting and date comparison	Compute in code; ask Jev only for the judgment
Reads instructions literally	State the exact condition; split a fuzzy question into two literal ones
Leans toward the first Choice option	Shuffle option order in shadow runs and track disagreement
Adversarial text in the state can move the answer	A Jev score can only block; systems of record decide
Accuracy falls as unrelated state grows	Filter first; send only the fields a question reads
Cannot generate or extract a value	An LLM or parser extracts candidates; Jev picks among them
Early access	Rate limits are adjusting dynamically; zero retention is offered on enterprise plans; no SOC 2 report appears on TypeSafe's public pages as of 12 October 2026; pin `jev-1.13.0`

Of this benchmark

- **Cost only.** Accuracy must be measured on your own decisions. A third-party preprint (not peer reviewed) evaluates Jev on 37 datasets, including routing and moderation, against Qwen3.8-27B and Gemma-4-E4B. It reports Jev ahead of Qwen on 27 and of Gemma on all 37, with 95 to 99% on IMDB, SST-2, HellaSwag and ARC, and notes that binary thresholds need tuning. None of those datasets is your traffic.
- **Other baselines.** A self-hosted classifier or a free moderation endpoint can cover some of these decisions at low cost. This benchmark compares hosted LLM calls only.
- **Structured output overhead.** Forcing JSON through tool use adds a few hundred system tokens per call on some providers. That is excluded and would raise the baselines.
- **Thinking off.** Not every model may allow thinking to be fully disabled. Reasoning tokens would raise the baselines and favor Jev further.
- **Assumed token counts and cache hit rate.** Real traffic varies, and every count is a parameter.
- **Latency is vendor-measured.** The 70 to 500 ms end-to-end figure comes from TypeSafe's launch post.
- **Prices move.** All prices are as of 12 October 2026.

SECTION 7

7 Reproduce the benchmark

Code, policy tests and this report: **github.com/GOhaike/decision-layer-benchmark**.

The calculator is a single Python file with no dependencies. It is attached to this PDF (open the attachments panel in Adobe Acrobat or Firefox) and printed in full in Appendix B. A second attachment, `jev_billing_probe.py`, confirms how Jev bills the state with two calls costing well under a cent.

```
python3 benchmark/jev_cost_model.py             # or: uv run python benchmark/jev_cost_model.py
uv run --with requests python benchmark/jev_billing_probe.py   # asks for your key
sh policy/run_tests.sh
```

Edit the scenario block to match your own traffic and run it again. Verified on Python 3.9 and 3.13 with identical output. Every figure in this report is generated by importing the same file.

Sources

1. OpenAI API pricing, https://openai.com/api/pricing/
2. Claude API pricing, https://platform.claude.com/docs/en/about-claude/pricing
3. Gemini API pricing, https://ai.google.dev/gemini-api/docs/pricing
4. TypeSafe models, https://docs.typesafe.ai/models
5. OpenAI prompt caching, developers.openai.com/api/docs/guides/prompt-caching
6. Gemini context caching, ai.google.dev/gemini-api/docs/caching
7. Jev 1.13 jaggedness, docs.typesafe.ai/model-jaggedness/jev-1.13
8. Introducing System One models and Jev, typesafe.ai (latency)
9. Evaluating and benchmarking the System One model Jev, Deußer, Sparrenberg and Sifa, arXiv preprint 2609.37647
10. Open Policy Agent, Rego policy language, openpolicyagent.org

APPENDIX A

Calculator output

What `python3 benchmark/jev_cost_model.py` prints for the reference workload.

```
Decision Layer Benchmark v1.1. 100,000 requests a day for 30 days, 5 questions, prices as of 2026-10-12
Jev state billed once per request: $315 a month (once: $315, per question: $1,071)

1. Decision layer only, per month
Model                        Naive    Cached     Tuned     Jev  Tuned/Jev
GPT-5.6 Sol               $234,000  $158,175   $41,906    $315     133.0x
Claude Opus 5.5           $180,000  $115,920   $30,300    $315      96.2x
Gemini 3.1 Pro Preview     $93,600   $93,600   $29,400    $315      93.3x

GPT-5.6 Terra              $93,600   $63,270   $16,762    $315      53.2x
Claude Sonnet 5.5          $90,000   $57,960   $15,150    $315      48.1x
Gemini 3.8 Flash           $33,750   $33,750   $10,688    $315      33.9x

GPT-5.6 Luna                $9,360    $6,327    $1,676    $315       5.3x
Claude Haiku 5.5            $4,500    $2,984      $793    $315       2.5x
Gemini 3.1 Flash-Lite      $11,700   $11,700    $3,675    $315      11.7x

2. Whole stack per month (reply always on the provider's mid-tier model)
A  Naive: mid-tier decisions, no caching
   OpenAI    GPT-5.6 Terra             $139,200   $45,915   $93,285   67%
   Anthropic Claude Sonnet 5.5         $132,000   $42,315   $89,685   68%
   Google    Gemini 3.8 Flash           $49,500   $16,065   $33,435   68%
B  Typical: small-model decisions, cached
   OpenAI    GPT-5.6 Luna               $44,344   $38,332    $6,012   14%
   Anthropic Claude Haiku 5.5           $36,974   $34,305    $2,668    7%
   Google    Gemini 3.1 Flash-Lite      $27,450   $16,065   $11,385   41%
C  Tuned: one batched small-model call, cached
   OpenAI    GPT-5.6 Luna               $39,694   $38,332    $1,361    3%
   Anthropic Claude Haiku 5.5           $34,783   $34,305      $478    1%
   Google    Gemini 3.1 Flash-Lite      $19,425   $16,065    $3,360   17%

Prices, $ per 1M tokens: input / output / cached input
  GPT-5.6 Sol               5.00 / 30.00 / 0.500   OpenAI
  GPT-5.6 Terra             2.00 / 12.00 / 0.200   OpenAI
  GPT-5.6 Luna              0.20 /  1.20 / 0.020   OpenAI
  Claude Opus 5.5           4.00 / 20.00 / 0.200   Anthropic
  Claude Sonnet 5.5         2.00 / 10.00 / 0.100   Anthropic
  Claude Haiku 5.5          0.10 /  0.50 / 0.010   Anthropic
  Gemini 3.1 Pro Preview    2.00 / 12.00 / 0.200   Google
  Gemini 3.8 Flash          0.75 /  3.75 / 0.075   Google
  Gemini 3.1 Flash-Lite     0.25 /  1.50 / 0.025   Google
  Jev 1.13                0.042 / free  / none    TypeSafe

Sources
  OpenAI     https://openai.com/api/pricing/
  Anthropic  https://platform.claude.com/docs/en/about-claude/pricing
  Google     https://ai.google.dev/gemini-api/docs/pricing
  TypeSafe   https://docs.typesafe.ai/models
  OpenAI caching https://developers.openai.com/api/docs/guides/prompt-caching
  Google caching https://ai.google.dev/gemini-api/docs/caching
```

APPENDIX B

The calculator

jev_cost_model.py · file fingerprint (md5) 61a3affc28826a4976fd5c4302547113, to confirm your copy matches the one behind these figures.

```
"""Decision Layer Benchmark v1.1: agent decision costs with and without Jev.

Run:  python3 jev_cost_model.py   (or: uv run python jev_cost_model.py)
Edit the SCENARIO block to match your own traffic. Python 3.9+, no dependencies.

Prices are standard list prices per 1M tokens read from each provider's pricing
page on 12 October 2026. Batch APIs (50% off, asynchronous) are excluded because
an agent decides in real time. Output is priced with thinking off.
"""
from __future__ import annotations

VERSION = "1.1"
PRICES_AS_OF = "2026-10-12"

SOURCES = {
    "OpenAI": "https://openai.com/api/pricing/",
    "Anthropic": "https://platform.claude.com/docs/en/about-claude/pricing",
    "Google": "https://ai.google.dev/gemini-api/docs/pricing",
    "TypeSafe": "https://docs.typesafe.ai/models",
    "OpenAI caching": "https://developers.openai.com/api/docs/guides/prompt-caching",
    "Google caching": "https://ai.google.dev/gemini-api/docs/caching",
}

# model: provider, tier, input, output, cached input, cache write, cache storage per hour
# ($ per 1M tokens). Cache write is what a cache miss costs on the static prefix.
MODELS = {
    "GPT-5.6 Sol":              ("OpenAI",    "frontier", 5.00, 30.00, 0.50,  6.25,  0.0),
    "GPT-5.6 Terra":            ("OpenAI",    "mid",      2.00, 12.00, 0.20,  2.50,  0.0),
    "GPT-5.6 Luna":             ("OpenAI",    "small",    0.20,  1.20, 0.02,  0.25,  0.0),
    "Claude Opus 5.5":          ("Anthropic", "frontier", 4.00, 20.00, 0.20,  5.00,  0.0),
    "Claude Sonnet 5.5":        ("Anthropic", "mid",      2.00, 10.00, 0.10,  2.50,  0.0),
    "Claude Haiku 5.5":         ("Anthropic", "small",    0.10,  0.50, 0.01,  0.125, 0.0),
    "Gemini 3.1 Pro Preview":   ("Google",    "frontier", 2.00, 12.00, 0.20,  2.00,  4.50),
    "Gemini 3.8 Flash":         ("Google",    "mid",      0.75,  3.75, 0.075, 0.75,  0.50),
    "Gemini 3.1 Flash-Lite":    ("Google",    "small",    0.25,  1.50, 0.025, 0.25,  1.00),
}
JEV_INPUT_PRICE = 0.042  # Jev 1.13, $ per 1M input tokens; output is free
# Smallest cacheable prefix per provider. OpenAI and Anthropic from their prompt-caching guides;
# Google from its implicit-caching table (Gemini 3.1 Pro Preview and 3.8 Flash; Flash-Lite
# is not listed and is assumed the same).
CACHE_MIN_TOKENS = {"OpenAI": 1_024, "Anthropic": 512, "Google": 4_096}

# ---------------- SCENARIO: change these to match your traffic ----------------
REQUESTS_PER_DAY = 100_000
DAYS = 30
QUESTIONS = ["route", "prompt injection", "personal data", "policy", "next step"]
DECISION_CALLS = 4          # separate-call baseline: injection and PII share one guardrail call
SYSTEM_TOKENS = 1_500       # static instructions per decision call (cacheable)
REQUEST_TOKENS = 1_500      # the request and its recent context (changes every request)
QUESTION_TOKENS = 200       # one question's instructions and options
OUTPUT_PER_CALL = 150       # separate call: JSON label plus a short reason
TERSE_OUTPUT_PER_QUESTION = 30  # batched call: terse JSON per question
CACHE_HIT_RATE = 0.95       # share of static-prefix reads served from cache
REPLY_SYSTEM_TOKENS = 1_500 # static part of the reply prompt (cacheable)
REPLY_DYNAMIC_TOKENS = 2_500
REPLY_OUTPUT_TOKENS = 600
JEV_BILLS_STATE_PER_QUESTION = False  # measured 12 Oct 2026: 2,011 tokens (1 question) vs 2,133 (5)
```

```
# ---------------------------------------------------------------------------

M = 1_000_000
HOURS = DAYS * 24
REQUESTS = REQUESTS_PER_DAY * DAYS


def price(model):
    return dict(zip(("provider", "tier", "inp", "out", "cached", "write", "storage"), MODELS[model]))


def can_cache(p, prefix_tokens):
    return prefix_tokens >= CACHE_MIN_TOKENS[p["provider"]]


def static_cost(p, tokens, cached):
    """Cost of a static prefix: full price uncached; hit or write price with caching."""
    if not cached:
        return tokens * p["inp"] / M
    return tokens * (CACHE_HIT_RATE * p["cached"] + (1 - CACHE_HIT_RATE) * p["write"]) / M


def storage(p, tokens):
    return tokens * p["storage"] * HOURS / M  # per month, for the whole cache


def decisions(model, mode):
    """Monthly cost of the decision layer on one model. mode: naive, cached, optimized."""
    p = price(model)
    if mode in ("naive", "cached"):
        cached = mode == "cached" and can_cache(p, SYSTEM_TOKENS)
        per_call = (static_cost(p, SYSTEM_TOKENS, cached) + REQUEST_TOKENS * p["inp"] / M
                    + OUTPUT_PER_CALL * p["out"] / M)
        month = per_call * DECISION_CALLS * REQUESTS
        return month + (storage(p, SYSTEM_TOKENS * DECISION_CALLS) if cached else 0)
    n = len(QUESTIONS)
    static = SYSTEM_TOKENS + n * QUESTION_TOKENS
    cached = can_cache(p, static)
    per_req = (static_cost(p, static, cached) + REQUEST_TOKENS * p["inp"] / M
               + n * TERSE_OUTPUT_PER_QUESTION * p["out"] / M)
    return per_req * REQUESTS + (storage(p, static) if cached else 0)


def jev(per_question=None):
    per_question = JEV_BILLS_STATE_PER_QUESTION if per_question is None else per_question
    n = len(QUESTIONS)
    tokens = n * (REQUEST_TOKENS + QUESTION_TOKENS) if per_question else REQUEST_TOKENS + n * QUESTION_TOKENS
    return tokens * JEV_INPUT_PRICE / M * REQUESTS


def reply(model, cached):
    p = price(model)
    cached = cached and can_cache(p, REPLY_SYSTEM_TOKENS)
    per = (static_cost(p, REPLY_SYSTEM_TOKENS, cached) + REPLY_DYNAMIC_TOKENS * p["inp"] / M
           + REPLY_OUTPUT_TOKENS * p["out"] / M)
    return per * REQUESTS + (storage(p, REPLY_SYSTEM_TOKENS) if cached else 0)


STACKS = [
    # name, decision tier, decision mode, reply cached
    ("A  Naive: mid-tier decisions, no caching", "mid", "naive", False),
    ("B  Typical: small-model decisions, cached", "small", "cached", True),
    ("C  Tuned: one batched small-model call, cached", "small", "optimized", True),
]
```

```
def stack(provider, tier, mode, cached):
    dmodel = next(m for m, v in MODELS.items() if v[0] == provider and v[1] == tier)
    rmodel = next(m for m, v in MODELS.items() if v[0] == provider and v[1] == "mid")
    without = decisions(dmodel, mode) + reply(rmodel, cached)
    with_jev = jev() + reply(rmodel, cached)
    return dmodel, rmodel, without, with_jev


def money(x):
    return f"${x:,.0f}"


def main():
    n = len(QUESTIONS)
    mode = "per question" if JEV_BILLS_STATE_PER_QUESTION else "once per request"
    print(f"Decision Layer Benchmark v{VERSION}. {REQUESTS_PER_DAY:,} requests a day for {DAYS} days, "
          f"{n} questions, prices as of {PRICES_AS_OF}")
    print(f"Jev state billed {mode}: {money(jev())} a month "
          f"(once: {money(jev(False))}, per question: {money(jev(True))})")

    print("\n1. Decision layer only, per month")
    print(f"{'Model':24}{'Naive':>10}{'Cached':>10}{'Tuned':>10}{'Jev':>8}{'Tuned/Jev':>11}")
    for tier in ("frontier", "mid", "small"):
        for model in [m for m, v in MODELS.items() if v[1] == tier]:
            nv, cv, tv = (decisions(model, k) for k in ("naive", "cached", "optimized"))
            print(f"{model:24}{money(nv):>10}{money(cv):>10}{money(tv):>10}{money(jev()):>8}"
                  f"{tv / jev():>10.1f}x")
        print()

    print("2. Whole stack per month (reply always on the provider's mid-tier model)")
    for name, tier, dmode, cached in STACKS:
        print(name)
        for provider in ("OpenAI", "Anthropic", "Google"):
            dmodel, rmodel, without, with_jev = stack(provider, tier, dmode, cached)
            saved = without - with_jev
            print(f"   {provider:10}{dmodel:24}{money(without):>10}{money(with_jev):>10}"
                  f"{money(saved):>10}{100 * saved / without:>5.0f}%")

    print("\nPrices, $ per 1M tokens: input / output / cached input")
    for model, (prov, _, pi, po, pc, _, _) in MODELS.items():
        print(f"  {model:24}{pi:>6.2f} / {po:>5.2f} / {pc:>5.3f}   {prov}")
    print(f"  {'Jev 1.13':24}{JEV_INPUT_PRICE:>6.3f} / free  / none    TypeSafe")
    print("\nSources")
    for provider, url in SOURCES.items():
        print(f"  {provider:10} {url}")


if __name__ == "__main__":
    main()
```